

Generative AI

Assignment 1: Latent Spaces and Why GANs Are Hard to Train

AE · VAE · vanishing gradient · mode collapse · DCGAN · IS and FID

Platform: Google Colab free tier. Every run in this assignment is sized to finish inside one session. **Starter:** `A1_starter.ipynb` on Classroom.

1. The point of this assignment

In class you were told that an autoencoder is not a generative model, that a VAE is, and that a GAN is powerful but temperamental. This assignment makes you produce the evidence yourself. By the end you should have, in your own figures:

- a VAE that walks smoothly from one face to another, and an AE decoder that turns a draw from $N(0, I)$ into nothing at all — with the measurement that says why;
- a generator whose gradient you can watch die, and the same generator rescued by changing one line of the loss;
- a generator that prints the same picture over and over, with a number attached to how bad it is;
- a generator whose samples you would be willing to show someone, and IS and FID values that agree with your eyes — except in one case, which you will explain.

Negative results are results. A stabiliser that did not help, reported honestly with evidence, earns full marks. A plot that has clearly been made to look like the lecture slides rather than like your run earns zero for that part.

1.1 Rules

- **Seed = the last four digits of your roll number**, set for torch, numpy and random at the top of every run. Your curves will not match anyone else's.
- **Three things are assigned by your roll number.** See §1.2. Using someone else's assignment is the easiest way to be caught.
- **Write your own training loops, and your own IS and FID.** PyTorch, torchvision, NumPy, scikit-learn, matplotlib and SciPy are fine. `pytorch-fid`, `clean-fid` or any package that implements the metric for you is not — you may use them at the end to check your own implementation, and say so.
- **Every training run writes a CSV.** Submit them. Several questions ask for a specific iteration number that only exists in your own log.
- **LLM use is allowed for debugging and for explaining concepts, and must be declared** in a short section of your report. Code you cannot explain at the viva is not your code, whatever produced it.

1.2 Your three assigned values

Let R = the last two digits of your roll number.

- **Held-out digit** $a = R \bmod 10$ — the anomaly class in Part B.
- **Your six MNIST digits** for Parts C–E: $D = \text{sorted}(\{ (R + 3k) \bmod 10 : k = 0..5 \})$. This always yields exactly six distinct digits.
- **Your second CelebA attribute** in Part A: index $R \bmod 6$ into [Eyeglasses, Male, Young, Blond_Hair, Wearing_Hat, Mouth_Slightly_Open]. Everyone does Smiling as well.

State all three on the first line of your report.

2. Part A — An AE's latent space is not a space (12 marks)

Dataset: CelebA, centre-cropped to 148×148 and resized to 64×64, 30 000 images is enough. Use the attribute file to find images with Smiling = +1 and Smiling = -1.

Build one encoder–decoder architecture and use it twice: once as a plain AE, once as a VAE. Same layers, same latent width $d = 64$, same optimiser, same epoch count. The only differences are the reparameterisation, the two heads (μ and $\log \sigma^2$) and the KL term. If you change anything else, the comparison is not a comparison.

A1 — Train both (2 marks)

- AE with MSE reconstruction loss. VAE with the Loss from lecture: reconstruction + $\frac{1}{2} \Sigma (\sigma^2 + \mu^2 - 1 - \log \sigma^2)$.
- **Report** final train and validation reconstruction MSE for both, and an 8-column grid of originals over reconstructions for each. Save as `figures/A1_recon.png`.
- The VAE reconstructions will be blurrier. Say by how much, in MSE, rather than saying "blurrier".

A2 — What the VAE can do: interpolate (3 marks)

A VAE learns a distribution over the latent space, so the points between two codes are themselves codes of plausible images. That is the property this task makes you see.

- **Pick one clearly smiling image and one clearly not-smiling image.** Encode both and interpolate the **means** (not a sample from q): $z_t = (1-t) \mu_1 + t \mu_2$ for eleven values $t = 0.0, 0.1, \dots, 1.0$. Decode each one.
- **Save as** `figures/A2_vae_interp.png`. Repeat for your assigned second attribute.
- **Two questions to answer from your own strip:** is every intermediate image a plausible face, and does the attribute change gradually along the strip or jump at one point? Say which, and show the frame where it happens.

A3 — What the AE cannot do: generate (3 marks)

An autoencoder has no distribution over its latent space. It only ever learned to decode the codes its own encoder produced. This task shows what that costs.

- **Draw 16 codes from $N(0, I)$ and decode them with the AE decoder, then with the VAE decoder.** Same 16 codes for both. Show the two grids side by side and save as `figures/A3_prior_samples.png`.
- **The VAE grid should contain recognisable faces and the AE grid should not.** Do not explain it yet — A4 is where the explanation earns marks.
- **For completeness, also decode 16 codes taken from the AE's own encoder** so the reader can see that the same decoder works perfectly well on the codes it was trained on. The decoder is not broken; the codes are in the wrong place.

3. Part B — What the AE is actually good for (3 marks)

Same conclusion, opposite sign: an AE is a poor generator and a good compressor. Both tasks here are on MNIST and take minutes.

B1 — Denoising (3 marks)

- **Train an AE with noisy inputs and clean targets:** $\tilde{x} = \text{clip}(x + 0.3 \cdot \epsilon, 0, 1), \epsilon \sim N(0, I)$.
- **Report** mean test PSNR of the noisy images against clean, and of the denoised against clean. One number each, plus a three-row grid of images (clean / noisy / denoised).
- One sentence: why is a model that cannot generate a face able to remove noise from one?

4. Part C — How a GAN fails (14 marks)

From here on, work on MNIST restricted to your six digits from §1.2, at 28×28, with z -dim 64 unless a question says otherwise. A small DCGAN is enough; the starter notebook gives you the shapes.

Log to CSV on every iteration: iteration, d_{loss} , g_{loss} , mean $D(x)$, mean $D(G(z))$, and the L2 norm of the generator's gradient. The starter has the hook; you must not delete it.

C1 — Watch the gradient die (5 marks)

Train with the **saturating** generator loss — the one from the original paper, $L_G = \log(1 - D(G(z)))$ — and make the discriminator too strong on purpose: five D steps per G step, and D's learning rate four times G's.

- **Plot $D(x)$ and $D(G(z))$ against iteration on one axis, and $\|\nabla_{\theta} G\|$ on a second plot with a log y-axis.**
- **Report the iteration at which your generator gradient norm first drops below 1% of its value at iteration 100.** Quote the two numbers. This value is specific to your seed; a report without it loses these marks.
- Show a 64-sample grid at that iteration and at the end of training.

C2 — Fix it with one line (4 marks)

- **Rerun with the non-saturating loss** $L_G = -\log D(G(z))$, same seed, same architecture, same D schedule. Nothing else may change.
- **Overlay the two gradient-norm curves on one figure** and show the two sample grids at the same iteration side by side. Save as `figures/C2_overlay.png`.
- **Three sentences** on the mechanism, using the two gradient expressions derived in class. The question to answer is why the two losses have the same fixed point but not the same gradient where it matters.

C3 — Make it collapse, then measure the collapse (5 marks)

Eyeballing a grid is not a measurement. Force a collapse and then quantify it.

- **Recipe:** give the generator **five steps for every discriminator step** at $lr\ 2e-3$ against D's $2e-4$, with z-dim 2 and no BatchNorm in G. G races ahead of a stale D, finds the one sample that fools it, and puts every z on that sample. Train until the samples stop changing.
- If your run does not collapse, tune it until it does and say what you changed. Note that a partial collapse — most of the mass on two or three classes — is the more realistic failure and is worth the same marks as a total one, provided you measure it.
- **Train the small classifier** in the starter on your six digits (it takes about a minute and reaches roughly 99% on this subset). Classify 5 000 generated samples.
- **Report three things:** the class histogram; the number of modes covered, meaning classes holding at least 1% of samples; and the reverse KL $D_{KL}(p_{gen} \parallel p_{data})$ against the uniform target over your six digits.
- Include a 64-sample grid. If your collapse is partial rather than total, say so — partial collapse with honest numbers is worth more than a claim of total collapse.

5. Part D — A GAN that actually works (10 marks)

By 2016 the recipe was known. Apply it.

D1 — DCGAN (6 marks)

- Strided convolutions instead of pooling, BatchNorm in both networks, LeakyReLU(0.2) in D, ReLU in G, tanh output, Adam($2e-4$, $\beta_1 = 0.5$), non-saturating loss, one D step per G step.
- **Train until the samples are recognisable digits.** Show a fixed-noise grid at four points in training — early, a third of the way in, two thirds, and the end — from the *same* z. That progression is the figure that matters.
- **Re-run the C3 measurement** (histogram, modes covered, reverse-KL) on this model.

D2 — One stabilizer, honestly reported (4 marks)

- **Choose one:** WGAN-GP, spectral normalization on D, or minibatch discrimination in D. Implement it; do not import it.
- **Fill in this table for all three models** — C3 collapsed, D1 DCGAN, D2 stabilized — with columns: modes covered, reverse-KL, IS, FID.
- **If the stabilizer did not improve things, say so and show the evidence.** Full marks. Two sentences on why you think it did not help are worth more than a number you wish you had.

One correction to a common mistake: minibatch discrimination goes in the **discriminator**, not the generator. It gives D a view of how similar the samples in a batch are to each other, so a generator that emits the same image is caught immediately. Putting it in G does nothing.

6. Part E — Measuring it: IS and FID (6 marks)

Implement both from the formulas, using the classifier you trained in C3: its softmax gives $p(y|x)$ for IS, its penultimate layer gives the features for FID. Real Inception-V3 is not needed on MNIST and is not wanted here — the point is that you can write the metric.

E1 — Inception Score (2 marks)

$IS = \exp(\mathbb{E}_x [D_{KL}(p(y|x} \parallel p(y))])$, with $p(y)$ the marginal over your generated set.

- Report mean $\pm$ std over 10 splits of 5 000 samples, for: real data, your C1 model, your C3 collapsed model, and your D1 and D2 models.

E2 — Fréchet Distance (2 marks)

$$\text{FID} = \| \mu_r - \mu_g \|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{\frac{1}{2}})$$

- **The matrix square root is the awkward part.** `scipy.linalg.sqrtm` on the product is the standard route; take the real part and say why a small imaginary component appears.
- Report FID for the same five sets as E1, against the real training set as reference.

E3 — Two sanity checks and one question (2 marks)

- **The floor.** Split the real test set into two disjoint halves and compute FID between them. Report it. It will not be zero — explain in one sentence what it is measuring.
- **Monotonicity.** Compute FID between the real set and the same images with Gaussian noise at $\sigma = 0, 0.1, 0.25, 0.5, 1.0$. Plot FID against σ . If your curve is not monotone, your implementation has a bug.
- **The question.** Your collapsed model from C3 may well score a respectable IS and a terrible FID. Using your own two numbers, explain in five sentences or fewer what each metric is blind to. This is the most important paragraph in the assignment.

7. Report, deliverables and marking

Four pages maximum, figures included. Two sections are compulsory and are read closely:

- **What surprised you.** Two things that did not happen the way the lecture led you to expect, with the figure that shows it.
- **What you could not get to work.** A run that diverged, a metric that came out wrong, a hyperparameter you could not tune. Say what you tried.

File	Contents
A2_<roll>.ipynb	One notebook, run top to bottom, all outputs saved. Parts A–F as marked sections.
logs/*.csv	One CSV per training run: iteration, d_loss, g_loss, D(x), D(G(z)), g_grad_norm.
figures/*.png	Every figure referred to in the report, named as in §2–§7.
metrics.json	The numbers you quote: MSE, PSNR, modes covered, reverse-KL, IS, FID.
report.pdf	Four pages maximum.

8. A note on getting help

You will be tempted to have a model write the training loops. Most of this assignment does not care — the loops are standard and you have seen them. What the marks are actually on is the part no one else can produce for you: the iteration at which your gradient died, the shape of your histogram, the number your FID floor came out at, and whether your explanation matches your own figures rather than a general account of how VAEs work. Come to the viva able to open your CSV and point at the row.

If you are stuck, the order to try things in is: read your own loss curves, print the shapes, overfit ten images to check the model can learn at all, and only then change the architecture.